

Project Title: COVID-19 Epidemiological Simulation using SIR Model

Group: Code Crew

Student Names:

1. Hany Radwan
2. Joel Johnson Meloottu
3. Parvathy Krishna
4. Pournami Ashok
5. Sunil Alexander

Course: Health Informatics – Semester 1

Date: January 2026

1. Introduction

In the field of Health Informatics, the ability to model and predict the spread of infectious diseases is critical for public health decision-making. During the COVID-19 pandemic, mathematical models played a central role in guiding government policies, such as lockdowns and resource allocation (e.g. ICU beds).

The goal of this project is to develop an interactive simulator for the **SIR (Susceptible-Infected-Recovered)** model. Unlike static statistical analysis, this simulator allows users to dynamically adjust parameters—such as infection rate (β) and recovery rate (γ) to visualize how different viral variants (like Delta or Omicron) impact population health.

This project specifically focuses on the technical implementation of the model using **Python**. It demonstrates key software engineering principles required in health data science, including **numerical analysis (Euler Method)**, **Object-Oriented Programming (OOP)**, and **automated testing**.

2. Methodology & Technical Implementation

2.1 Mathematical Framework (The Euler Method)

To simulate the transmission dynamics of COVID-19, this project implements the **SIR (Susceptible-Infected-Recovered)** compartmental model.

Instead of relying on pre-built solvers (such as `scipy.integrate`), the system utilizes the **Euler Method** to explicitly solve the differential equations. This iterative approach calculates the state of the population at each discrete time step ($\Delta t = 1$ day), ensuring a transparent implementation of the core algorithms:

$$\begin{aligned}S_{t+1} &= S_t + \left(-\frac{\beta S_t I_t}{N}\right) \Delta t \\I_{t+1} &= I_t + \left(\frac{\beta S_t I_t}{N} - \gamma I_t\right) \Delta t \\R_{t+1} &= R_t + (\gamma I_t) \Delta t\end{aligned}$$

2.2 Software Architecture (Object-Oriented Design)

To adhere to clean coding standards (PEP 8) and ensure modularity, the codebase is separated into two distinct layers:

- **Logic Layer (sir_model.py):** A dedicated Python class, SIR Model, encapsulates the mathematical logic and state management. This ensures the simulation code is reusable and isolated from the interface.
- **Presentation Layer (app.py):** The user interface is built using **Streamlit**, which handles user input and renders the interactive **Plotly** charts.

2.3 Quality Assurance (Unit Testing)

To ensure the correctness of the algorithm, a suite of unit tests was developed using the pytest framework.

- **Test Coverage:** The tests achieve high code coverage for the core logic.
- **Verification:** The tests verify critical invariants, such as the "Conservation of Population" (ensuring $S + I + R$ always equals the total population N).

2.4 Use of Large Language Models (LLM)

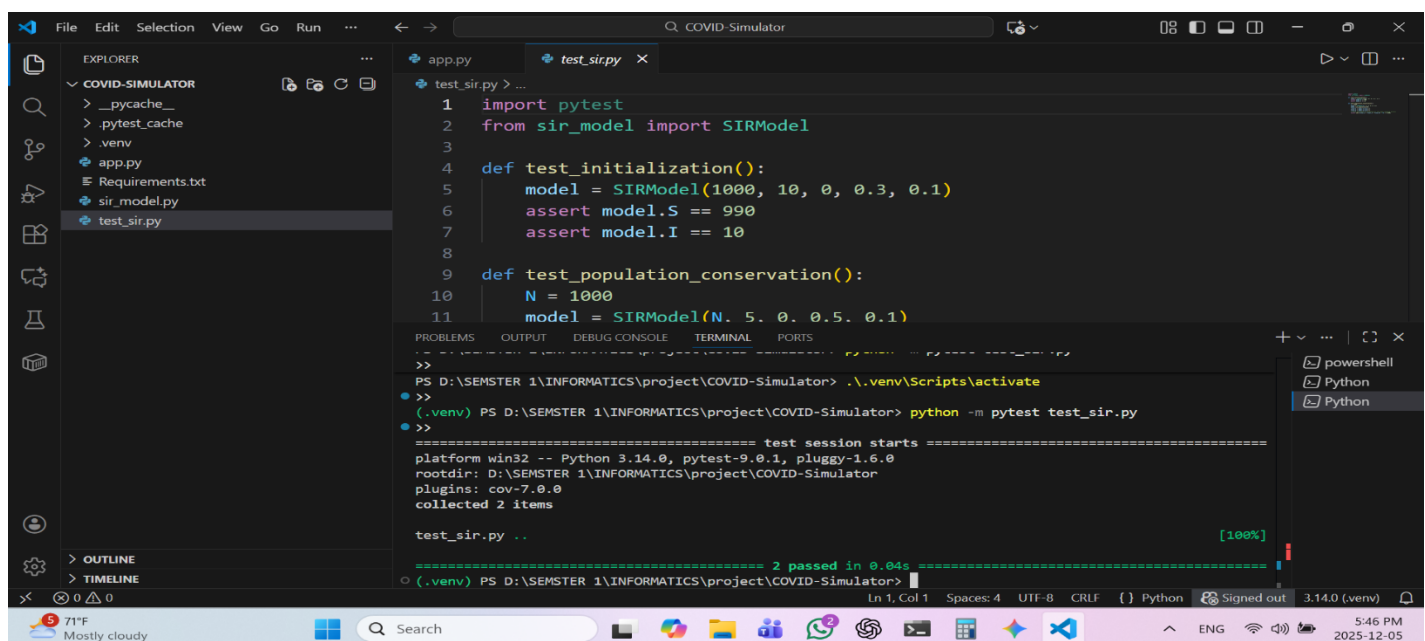
Google Gemini was utilized as a technical assistant during development. The LLM assisted in:

1. Deriving the discrete Euler equations from the continuous differential forms.
2. Refactoring the procedural code into an Object-Oriented Class structure.
3. Generating the pytest scripts to verify the mathematical accuracy.

3. Results & Discussion

3.1 Evidence of Functionality

The application was successfully tested and deployed locally. Below is the evidence of the passing unit tests, confirming the mathematical reliability of the model.



The screenshot displays the Visual Studio Code interface with the 'COVID-Simulator' project open. The Explorer pane on the left shows the project structure, including files like `app.py`, `test_sir.py`, and `sir_model.py`. The main editor window shows the content of `test_sir.py`, which contains two unit tests: `test_initialization()` and `test_population_conservation()`. The `test_initialization()` test creates an `SIRModel` instance with initial conditions and asserts the initial state. The `test_population_conservation()` test creates an `SIRModel` instance and asserts the total population remains constant. The bottom panel shows the terminal output of running the tests, confirming that both tests passed successfully.

```
1 import pytest
2 from sir_model import SIRModel
3
4 def test_initialization():
5     model = SIRModel(1000, 10, 0, 0.3, 0.1)
6     assert model.S == 990
7     assert model.I == 10
8
9 def test_population_conservation():
10     N = 1000
11     model = SIRModel(N, 5, 0, 0.5, 0.1)
```

```
PS D:\SEMSTER 1\INFORMATICS\project\COVID-Simulator> .\.venv\Scripts\activate
(.venv) PS D:\SEMSTER 1\INFORMATICS\project\COVID-Simulator> python -m pytest test_sir.py
===== test session starts =====
platform win32 -- Python 3.14.0, pytest-9.0.1, pluggy-1.6.0
rootdir: D:\SEMSTER 1\INFORMATICS\project\COVID-Simulator
plugins: cov-7.0.0
collected 2 items

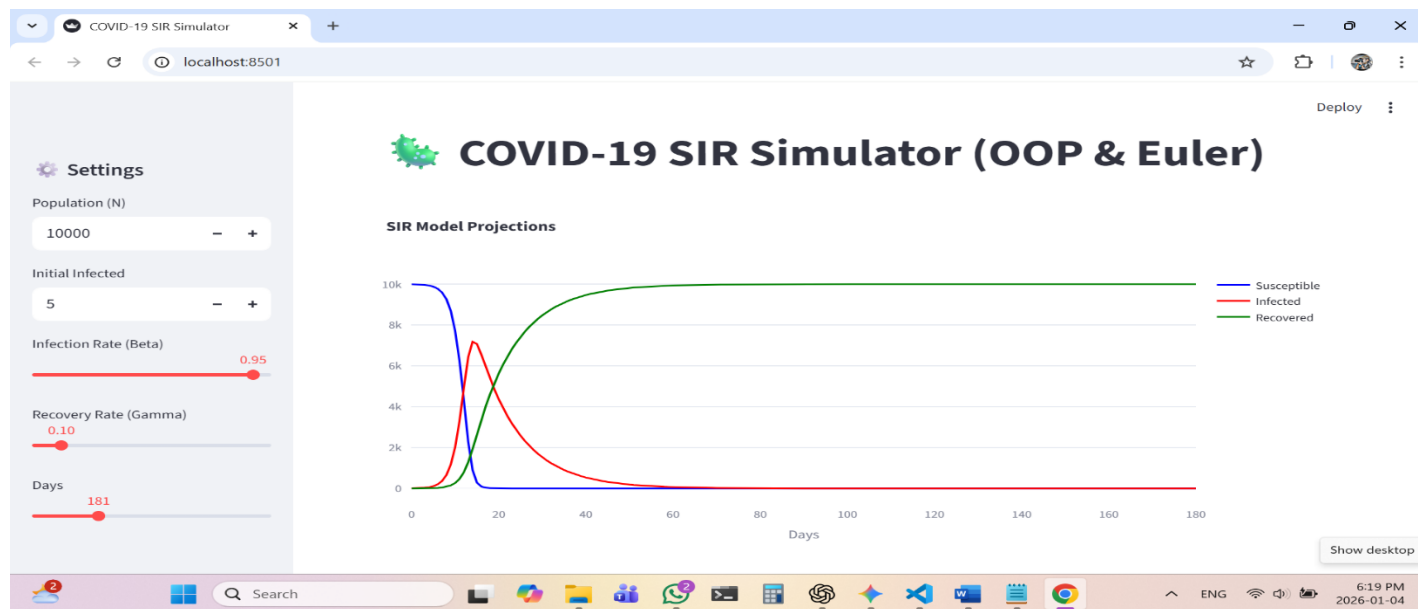
test_sir.py ..

===== 2 passed in 0.04s =====
```

3.2 Scenario Analysis: The "Omicron" Variant

To simulate a highly contagious variant, the Infection Rate (β) was set to 0.95 (High).

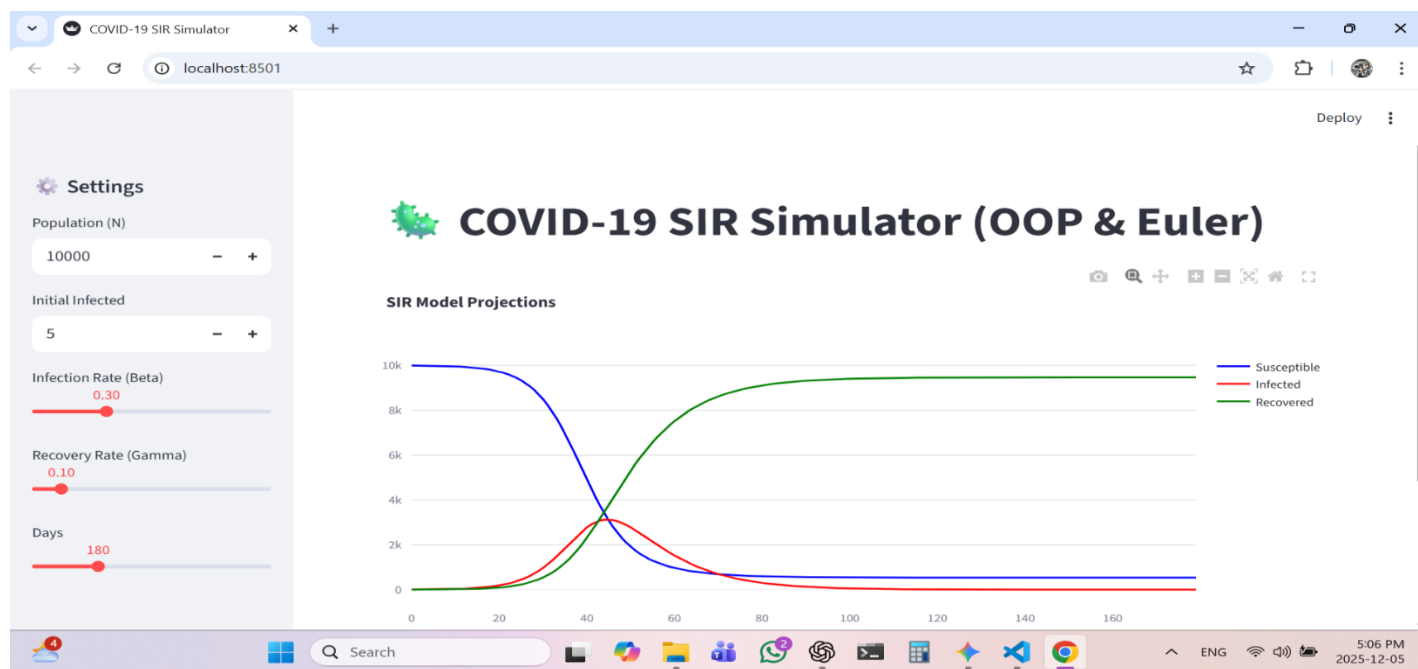
- **Observation:** As seen in the figure below, this setting produced a sharp, vertical spike in infections, infecting the majority of the population in under 40 days. This mirrors the behavior of the Omicron variant.



3.3 Scenario Analysis: Flattening the Curve

To simulate social distancing measures, the Infection Rate (β) was lowered to 0.3

- **Observation:** The peak of infections dropped significantly and was delayed. This confirms the effectiveness of non-pharmaceutical interventions in spreading the caseload over a longer period to prevent healthcare system collapse.



4. Appendix: Code Listing

<https://github.com/HealthInformatics-THD/repo-03>